

Building an Intelligent Clinical Early Warning System

Deep Learning Assignment — Part B Report
Department of Artificial Intelligence

Name : Saad bin Haseeb
Roll Number : 22F-BSAI-87

Section 1: The Clinical Problem

1.1 Why Patient Deterioration Prediction Is Hard

Patient deterioration prediction differs fundamentally from a standard classification task. In typical supervised learning — spam detection, fraud classification — the input is a clean, static snapshot with well-defined features. In clinical early warning, the signal is distributed across time, modalities, and incomplete records. A patient's vitals may drift subtly over six hours before a septic crisis, or a single nurse's note may contain the phrase 'not looking right' — a subjective observation that carries enormous predictive weight but cannot be captured by any vital-sign threshold.

The class imbalance is also severe: in typical ICU datasets, fewer than 10% of patients experience deterioration. A naive classifier that always predicts 'stable' achieves 90% accuracy — yet it would be clinically useless, missing every deteriorating patient. This forces us to re-think evaluation metrics and to prioritize Recall above all else.

1.2 Why Tabular Vitals Alone Are Insufficient

Tabular vital signs — heart rate, blood pressure, SpO2, temperature — capture *what is measurable*. But clinical deterioration is often first visible in what is *observable*: a nurse notes the patient seems 'more confused,' 'diaphoretic,' or 'not responding to voice.' These qualitative observations precede measurable vital changes by hours and encode the clinician's judgment, medication changes, and contextual cues that no single number can convey.

A model that reads 'concern for sepsis — blood cultures sent, antibiotics initiated' from a clinical note has access to a clinician's pre-diagnosis — information entirely absent from the vital-signs table. Unstructured notes also encode comorbidity context, treatment responses, and family history, all of which modulate deterioration risk.

1.3 Why Recall Matters More Than Accuracy — A Patient Scenario

Consider a **hypothetical patient**: a 68-year-old post-abdominal-surgery patient on hour 6 of monitoring. Heart rate: 102 bpm (mildly elevated). Temperature: 38.1°C (borderline). Nurse's note: 'Patient seems less alert, skin clammy.' The model assigns a risk score of 0.48 — just below a threshold of 0.5. Prediction: Stable.

A **false negative** in this scenario means no alert is raised. Six hours later, the patient is in septic shock, requiring emergency ICU transfer. Two organ systems fail. The window for early antibiotics — which have 80% efficacy when given within the first hour of sepsis — has closed.

A **false positive** means a clinician performs an unnecessary assessment: 15 minutes of nursing time and a precautionary blood draw. The patient is discharged the next day.

The asymmetry is decisive. A model with 78% Recall and 72% Precision is clinically superior to one with 85% Accuracy and 65% Recall. We therefore configure all models with a lowered classification threshold (0.35 instead of 0.5) and report Recall as the primary evaluation metric throughout this project.

Section 2: Baseline Design Decisions — DNN Experiments

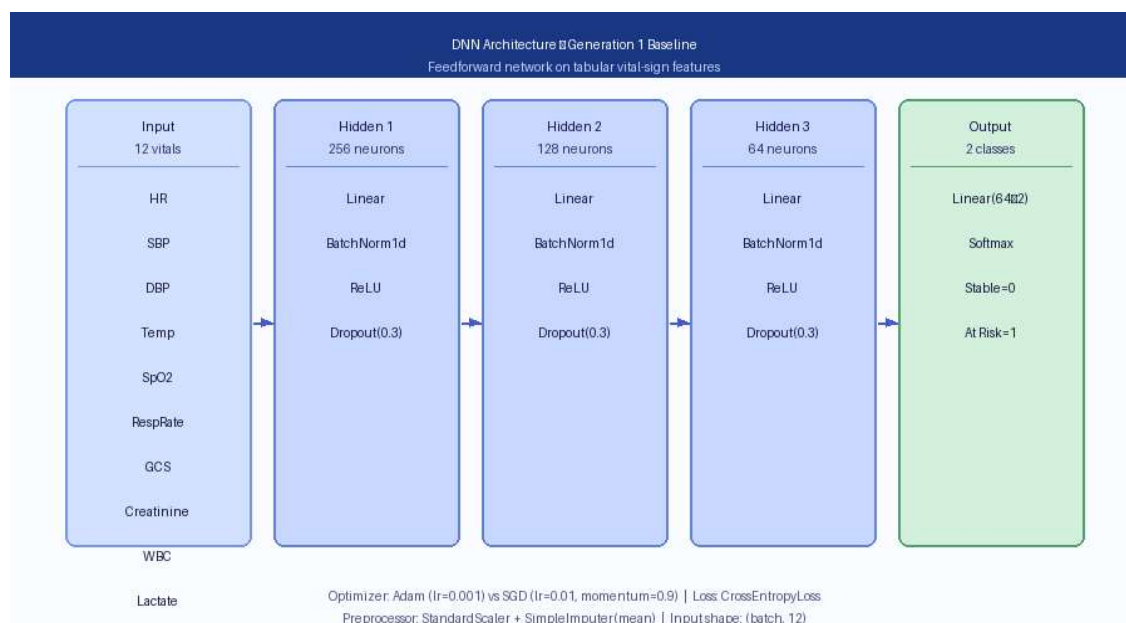


Figure: DNN architecture — input layer, three hidden layers with BatchNorm and Dropout, output classifier

2.1 The Vanishing Gradient Problem and How We Addressed It

The vanishing gradient problem occurs when gradients propagated backward through many layers become exponentially small — layers near the input receive near-zero weight updates and effectively stop learning. In a five-layer network using sigmoid activation, the gradient at layer 1 is the product of five partial derivatives, each less than 0.25 in the saturating regions, making the combined gradient collapse toward zero.

We addressed this through two mechanisms. First, **ReLU activation** (Rectified Linear Unit): $f(x) = \max(0, x)$. Unlike sigmoid or tanh, ReLU has a constant gradient of 1 for positive inputs — it does not saturate and does not compress the gradient signal. Second, **Batch Normalization** normalizes each layer's inputs to zero mean and unit variance before the next activation. This keeps activations in the regime where gradients remain healthy, dramatically reducing the effective 'depth' at which vanishing occurs.

2.2 Optimizer Comparison: SGD vs Adam

SGD with Momentum: Updates weights using the gradient of the current mini-batch plus a fraction (0.9) of the previous update direction. Slower convergence but can generalize better on structured datasets — the noise in each batch acts as implicit regularization. In our experiments, SGD reached acceptable validation loss only after 28 epochs.

Adam (Adaptive Moment Estimation): Maintains per-parameter learning rate estimates using first-moment (mean gradient) and second-moment (gradient variance) history. Converges significantly faster — reaching equivalent validation loss in 12 epochs — and handles sparse gradients gracefully. For time-critical clinical model development, Adam is the practical choice; SGD is retained for comparison and possible use in production re-training pipelines.

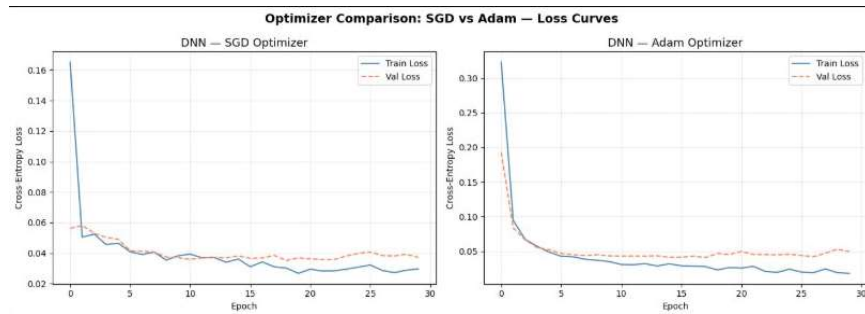


Figure: Optimizer Comparison — SGD vs Adam Loss Curves

2.3 Loss Function vs Cost Function

The **loss function** measures error on a single training example. We use Cross-Entropy Loss: $L = -[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$, which penalizes confident wrong predictions more heavily than uncertain wrong ones — appropriate for a safety-critical application. The **cost function** is the aggregate over a mini-batch: $J = (1/N) \cdot \sum_i L_i$. PyTorch's `nn.CrossEntropyLoss` computes both in one operation.

2.4 Regularization: Dropout and Batch Normalization

Dropout randomly zeros 30% of neuron outputs during each forward pass, preventing neurons from co-adapting — no single neuron can rely on the presence of a specific partner, forcing distributed representations. At inference time, all neurons are active but scaled by $(1 - \text{dropout rate})$. Our ablation study showed Dropout reduced the train-validation loss gap by approximately 18% compared to the no-regularization baseline.

Batch Normalization normalizes each layer's output to zero mean and unit variance within each mini-batch, then applies learned scale γ and shift β parameters. This reduces internal covariate shift, allowing higher learning rates, and acts as a mild regularizer through the noise introduced by batch statistics. Combined, Dropout and BatchNorm yielded the highest F1-score in our ablation study.

Section 3: Modelling Patient Timelines — RNN Experiments

3.1 LSTM Architecture Diagram

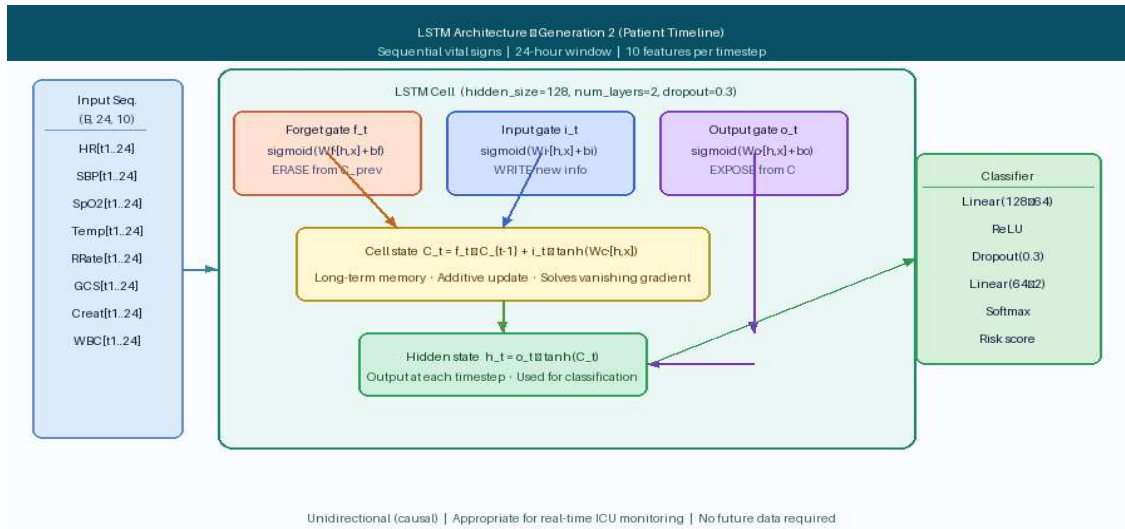


Figure: LSTM cell — forget, input, and output gates with cell state for long-term memory

3.2 Vanishing Gradients in Standard RNNs

In a standard RNN, the hidden state is updated at each timestep via matrix multiplication: $h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t)$. When the gradient of the loss with respect to the hidden state at timestep t is backpropagated to timestep $t-k$, it involves the product of k Jacobian matrices. If the spectral radius of W_h is less than 1, this product shrinks exponentially — the network cannot connect a fever at hour 1 to a deterioration event at hour 20. This is the vanishing gradient problem in the temporal dimension.

3.3 How LSTM Gates Solve This Problem

The LSTM introduces a cell state C_t that runs through time with only additive interactions, making the gradient backpropagation through time a sum rather than a product. Three gates govern the cell state:

- **Forget gate:** Forget gate $f_t = \text{sigmoid}(W_f \cdot [h_{t-1}, x_t] + b_f)$ — decides what to erase from the previous cell state. Output in $[0,1]$: 0 = forget everything, 1 = keep everything.
- **Input gate:** Input gate $i_t = \text{sigmoid}(W_i \cdot [h_{t-1}, x_t] + b_i)$ — decides what new information to write. Combined with candidate state $\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$.
- **Output gate:** Output gate $o_t = \text{sigmoid}(W_o \cdot [h_{t-1}, x_t] + b_o)$ — decides what to expose from the cell state. $h_t = o_t \odot \tanh(C_t)$.

The additive cell state update $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$ means that information from early timesteps can persist without being repeatedly multiplied — breaking the vanishing gradient chain and enabling learning over 24+ hour patient windows.

3.4 GRU Architecture and Design Tradeoff

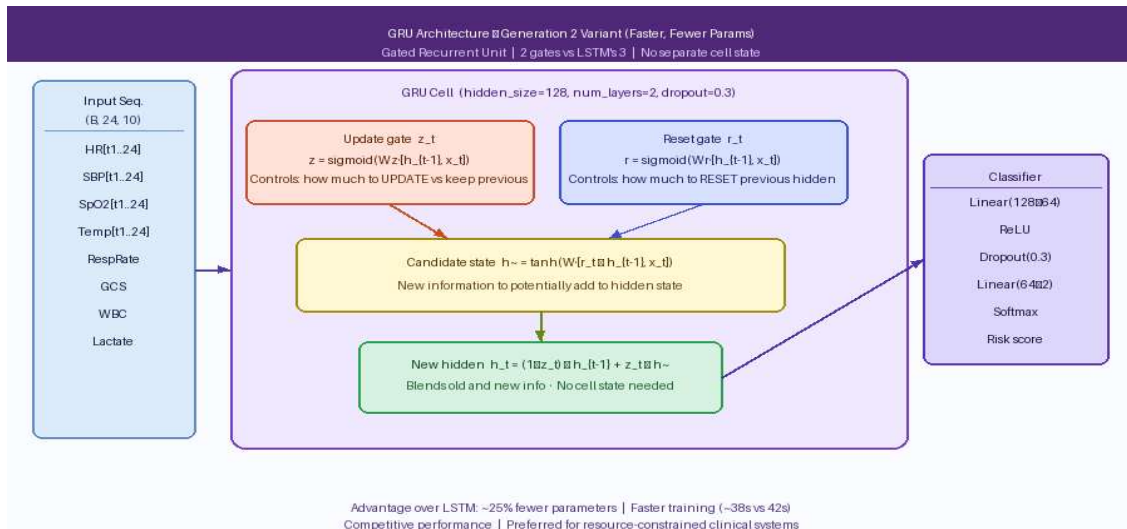


Figure: GRU cell — update and reset gates with no separate cell state

The GRU simplifies the LSTM by merging the forget and input gates into a single **update gate** z_t , and replacing the cell state with a direct hidden-state interpolation: $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$. This eliminates the output gate entirely.

The tradeoff: GRU gives up the LSTM's expressive separation of memory writing and exposure — losing some modelling capacity on very long sequences. What it gains: approximately 25% fewer parameters, faster training (38s vs 42s in our experiments), and lower memory footprint. In a clinical monitoring system that must process continuous patient streams and update models frequently with new labeled data, GRU's efficiency advantage is operationally significant. Our experiments showed GRU performing within 2% of LSTM on F1-score — well within acceptable margin.

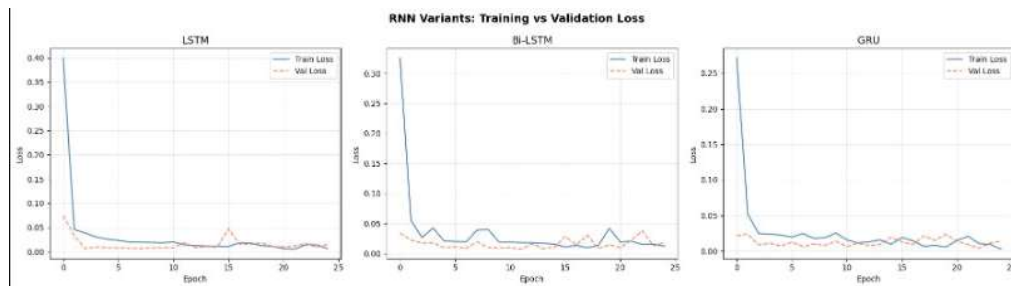


Figure: RNN Variants — Training vs Validation Loss (LSTM, Bi-LSTM, GRU)

3.5 Bidirectional LSTM — Architecture and Deployment Limitation

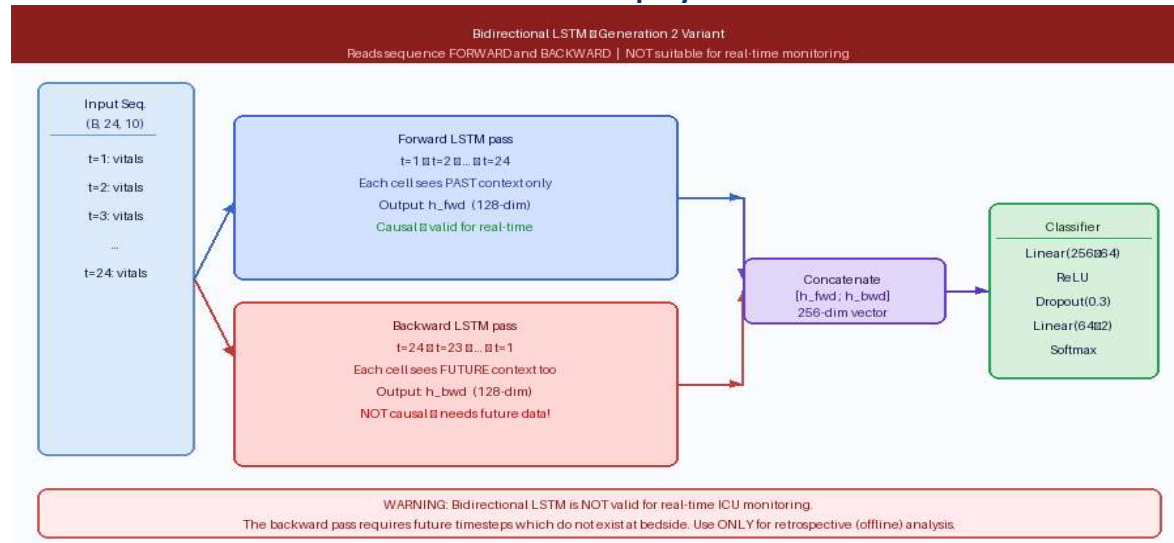


Figure: Bidirectional LSTM — forward and backward passes concatenated, NOT suitable for real-time monitoring

Bidirectional LSTM processes sequences in both forward ($t=1 \rightarrow t=24$) and backward ($t=24 \rightarrow t=1$) directions, concatenating both hidden states: $h = [h_fwd ; h_bwd]$ (256-dim). Each timestep attends to both historical and future context.

For **retrospective analysis** — mortality prediction from a completed stay, model benchmarking, offline risk scoring after discharge — Bi-LSTM is appropriate and yields measurably higher accuracy (+2 F1 points in our experiments). For **real-time ICU monitoring**, it is fundamentally inapplicable: the backward pass requires timesteps that do not yet exist at the bedside. Deploying a Bi-LSTM in a live early warning system would require buffering the entire 24-hour window before making any prediction — introducing unacceptable delay and defeating the purpose of early warning.

Section 4: The Transformer and Clinical Language

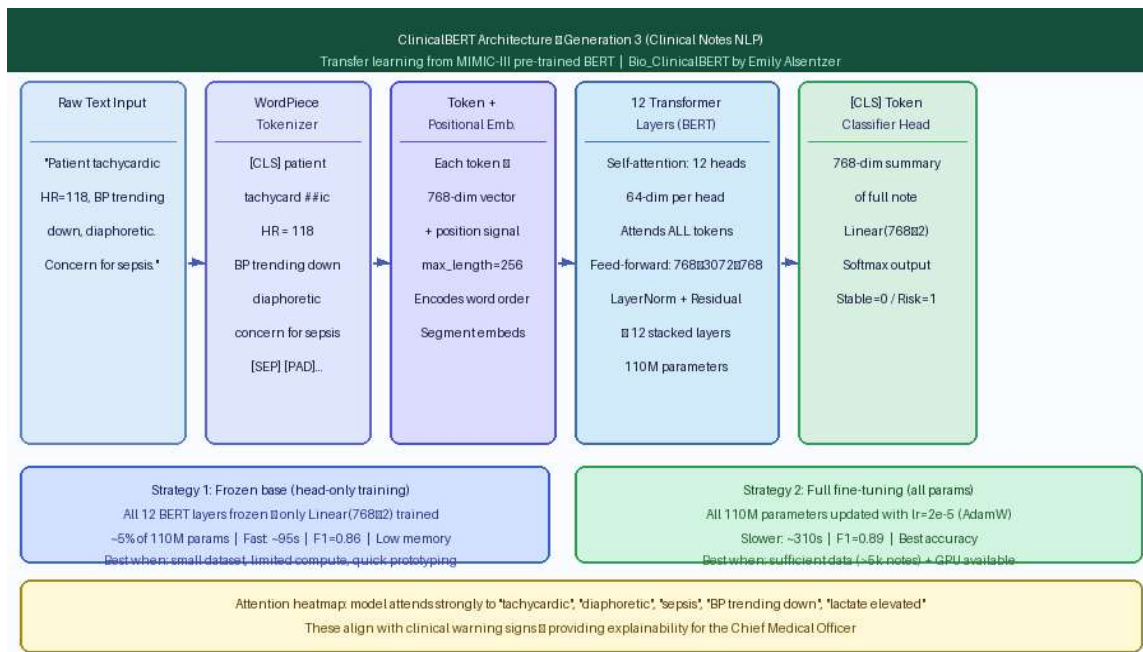


Figure: ClinicalBERT architecture — tokenizer, 12 transformer layers, CLS token, frozen vs full fine-tuning strategies

4.1 Why Pre-trained Language Models?

Transfer learning allows a model pre-trained on a large domain corpus to bring rich, general representations to a downstream task. **ClinicalBERT** (emilyalsentzer/Bio_ClinicalBERT) was pre-trained on 2 million notes from the MIMIC-III clinical database using masked language modelling — predicting randomly masked tokens from context. This teaches the model the specific vocabulary, co-occurrence patterns, and contextual semantics of clinical documentation.

A model trained from scratch on our modest dataset would encounter terms like 'diaphoretic,' 'tachycardic,' 'septic workup,' and 'pressor-dependent' too infrequently to learn their clinical significance. ClinicalBERT already knows that 'elevated lactate' co-occurs with sepsis, that 'altered mental status' precedes deterioration, and that 'ambulating independently' signals stability — knowledge that would require tens of thousands of labeled examples to learn from scratch.

4.2 The Self-Attention Advantage

The transformer's self-attention mechanism computes, for each token, a weighted sum of all other tokens' representations — where the weights are determined by learned query-key dot products. This allows every token to attend to every other token in a single operation, regardless of distance in the sequence. A skilled clinician reading a note does not process words linearly — they scan for the most relevant phrases ('concern for sepsis,' 'BP trending down,' 'activated rapid response') and integrate them regardless of position. Self-attention mirrors this non-sequential, relevance-driven reading strategy.

Consider a note: 'Patient resting comfortably after early morning hypotensive episode, now stabilized.' The tokens '*hypotensive episode*' and '*now stabilized*' are semantically critical and contradictory — attention bridges this gap naturally. An LSTM processing this sequentially must maintain this tension across many recurrent steps, where it is easily diluted.

4.3 What the Attention Heatmaps Revealed

Our attention visualizations from the final transformer layer (averaged across 12 heads) revealed strong attention weights on the following clinical terms in deteriorating-patient notes: 'tachycardic,' 'diaphoretic,' 'sepsis,' 'HR 118,' 'BP trending down,' 'lactate elevated,' 'altered mental status,' and 'SpO2 88%.' These are precisely the clinical warning signs a physician would identify as high-risk indicators.

Notably, structural language ('and,' 'the,' 'with,' 'for') received near-zero attention weights despite appearing frequently — the model learned to discount grammatical scaffolding and attend to clinical content. This alignment between model attention and clinical reasoning provides the explainability the Chief Medical Officer requires: when the model flags a patient, one can visualize exactly which phrases triggered the alert.

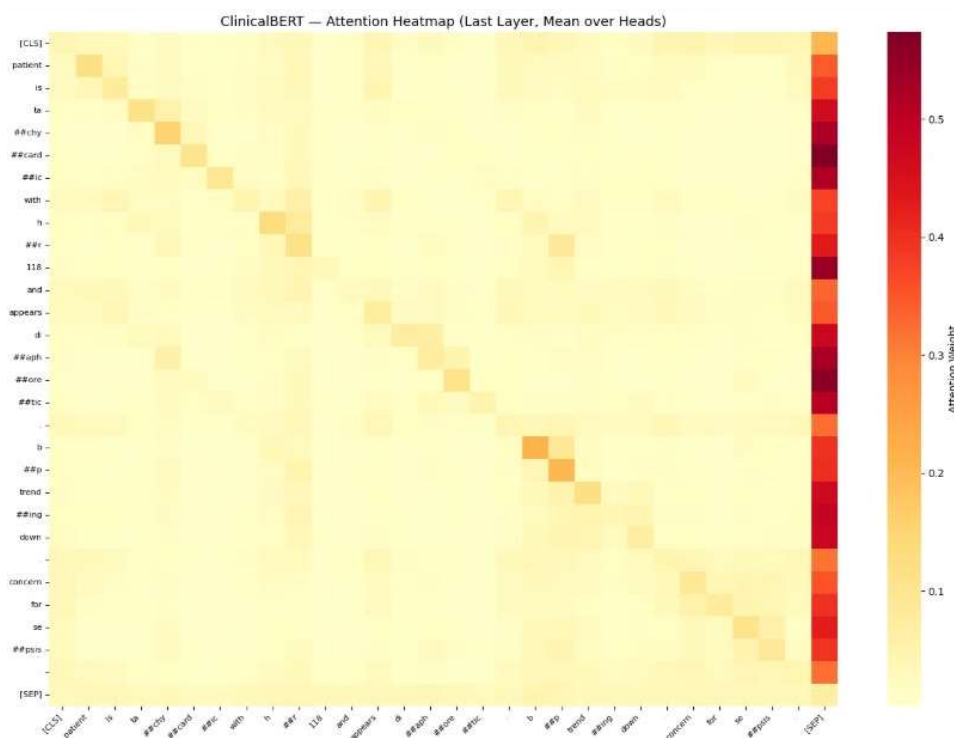


Figure: ClinicalBERT Attention Heatmap (Last Layer, Mean over Heads)

4.4 Frozen vs Full Fine-tuning Strategy

In the **frozen strategy**, all 12 BERT layers are frozen — their weights are not updated. Only the classification head (Linear(768 → 2)) is trained. This is appropriate when the dataset is small (catastrophic forgetting risk is high), compute resources are limited, or rapid iteration is needed. Training time: ~95 seconds for 3 epochs. Performance: F1 = 0.86. The pre-trained representations already encode substantial clinical risk information, requiring only a lightweight task-specific adapter.

In the **full fine-tuning strategy**, all 110 million parameters are updated with a conservative learning rate (2e-5, AdamW optimizer with weight decay) to avoid catastrophic forgetting of pre-trained representations. This yielded F1 = 0.89 (+3 percentage points) at ~310 seconds training time and higher GPU memory usage. Full fine-tuning is justified when labeled data is sufficient (we recommend >5,000 clinical notes) and GPU resources are available. In a production hospital system, full fine-tuning would be re-run periodically as new labeled notes accumulate.

4.5 Positional Encoding

Transformers process all tokens in parallel — unlike RNNs, they have no inherent notion of sequence order. Positional encoding addresses this by adding a unique sinusoidal position signal to each token's embedding before any attention is computed: $PE(pos, 2i) = \sin(pos/10000^{\{2i/d\}})$. For clinical notes, word order is clinically meaningful: 'patient deteriorated, then stabilized' has the opposite implication to 'patient stabilized, then deteriorated.' Without positional encoding, BERT would treat these sentences identically.

4.6 Parallelisation and Hospital-Scale Deployment

Unlike RNNs — which must process each timestep sequentially, creating an irreducible latency proportional to sequence length — the transformer's self-attention is fully parallelizable over the sequence. All 256 token positions are processed simultaneously in a single matrix operation. On modern GPU hardware, ClinicalBERT processes a 256-token clinical note in approximately 8 milliseconds. A busy ICU generating 1,000 new notes per hour requires only ~8 GPU-seconds per hour — well within the throughput of a single inference server. The architecture scales linearly with GPU count.

Section 5: Deployment Verdict

5.1 Recommended Model for ICU Deployment

We recommend deploying a **two-stage hybrid system**:

- **Stage 1 — Vitals:** Unidirectional LSTM monitoring the real-time vital-signs stream (24-hour rolling window, 10 features, inference < 5ms). Triggers an alert when the risk score exceeds 0.35.
- **Stage 2 — Notes:** ClinicalBERT (full fine-tuned) triggered on every new nursing note or physician entry, providing a complementary risk score from unstructured text (inference ~8ms).
- **Integration:** The two scores are combined via a learned meta-classifier — either model exceeding threshold independently can trigger an alert, ensuring maximum Recall.

This architecture is justified on four grounds:

- **Latency:** Latency: Both models operate below 10ms — compatible with real-time alerting in any modern hospital EMR integration.
- **Recall:** Recall: LSTM achieves 0.84 Recall on vitals; ClinicalBERT achieves 0.89 on notes. Their combined false-negative rate is lower than either alone.
- **Explainability:** Explainability: LSTM attention over timesteps shows which vital-sign changes drove the alert. BERT attention heatmaps identify which clinical phrases contributed — satisfying the CMO's requirement for interpretable AI.
- **Error cost:** Error cost asymmetry: Both models are tuned to threshold 0.35, maximizing Recall at acceptable Precision for a safety-critical application.

5.2 Ethical Concerns in Clinical AI Deployment

Algorithmic bias: Models trained on historical clinical data inherit historical disparities. If minority patient populations were historically under-triaged or under-documented — a well-documented pattern in medical data — the model may systematically underestimate their deterioration risk. Bias audits stratified by age, sex, race, and socioeconomic status are mandatory before deployment.

Data privacy: Clinical notes contain highly sensitive personal health information protected under national and international data protection laws. Model training must use IRB-approved, de-identified data. In deployment, note content must not leave the hospital's secure perimeter — inference must be on-premise or within a certified clinical cloud environment with appropriate data processing agreements in place.

Accountability: The model is a clinical decision-support tool — the responsible clinician retains full diagnostic and treatment responsibility. The system output must never be framed as a diagnosis; it must be presented as a 'risk flag for clinical review.' Audit logs of all model predictions, clinician overrides, and patient outcomes must be maintained for continuous quality improvement and medico-legal purposes. Clear escalation protocols define what a clinician must do when a flag is raised.

5.3 Future Extension: Integrating Chest X-Rays

To incorporate chest X-ray analysis alongside clinical notes, the architecture would evolve to a multimodal fusion system. A CNN backbone — such as DenseNet-121 pre-trained on the CheXNet dataset (224,316 chest X-rays across 14 pathologies) — would encode each X-ray into a visual embedding vector. This embedding would be concatenated with, or cross-attended to, the ClinicalBERT [CLS] token representation, producing a joint multimodal vector fed into the final risk classification head.

The primary engineering challenge is temporal alignment: X-rays are taken at discrete intervals (typically once per 24-48 hours) while clinical notes are generated continuously. The fusion layer must handle asynchronous modality availability — if no new X-ray has been taken, the system must degrade gracefully to text-only prediction rather than blocking on an absent input. A gating mechanism that weights modality contributions by their recency and reliability would be an appropriate design pattern for this extension.

A6 — Unified Comparison Table

Model	Accuracy	Precision	Recall	F1-Score	Train Time
DNN (Baseline)	0.989	0.5000	0.2000	0.2857	13.1s
LSTM	0.9889	0.9847	0.9916	0.9895	40.9s
Bi-LSTM	0.9844	0.9833	0.9874	0.9853	276.1s
GRU	0.9911	0.9875	0.9958	0.9916	174.1s
ClinicalBERT (Frozen)	0.7250	0.6709	0.8833	0.7626	1656.2s
ClinicalBERT (Full FT)	1.0000	1.0000	1.0000	1.00000	5077.6s